

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

RUBRICA DE EVALUACIÓN — EV-AUT-03

Portafolio de Evidencias Individuales

Autoevaluación sumativa final del período: tres evidencias de aprendizaje y contribución individual verificable al Proyecto Fin de Curso

Código del instrumento:	EV-AUT-03
Tipo y Carácter:	Evaluación (EV) — Autoevaluación Sumativa
Asignatura:	Aplicaciones Distribuidas (ISR-701)
Estudiante:	Jeremy Alexis Alvarez Párraga
Equipo PFC (sigla):	D / ACC (ACC — Soporte Técnico ISP)
Usuario de GitHub:	DeJere (https://github.com/DeJere)
Correo institucional:	jalvarezp3@uteq.edu.ec
Docente responsable:	Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico:	2026–2027 PPA
Fecha de evaluación:	21 de agosto de 2026

URL del repositorio del portafolio (única línea):

<https://github.com/DeJere/Aplicaciones-distribuida-Alvarez-P-rraga-.git>

URL del repositorio del PFC (única línea):

<https://github.com/DeJere/Aplicaciones-distribuida-Alvarez-P-rraga-.git>

1. Propósito del Instrumento y Objeto de la Evaluación

El presente portafolio individual de evidencias operativiza la actividad sumativa **EV-AUT-03** de la asignatura **Aplicaciones Distribuidas (ISR-701)**. En estricta correspondencia con las directrices de la rúbrica analítica, este instrumento evalúa mi capacidad individual para identificar, documentar y justificar mi propio proceso de aprendizaje a través de artefactos técnicos reales, verificables y rastreables en el historial de versiones de mi repositorio en GitHub [1], [2].

El objeto evaluado es lo que yo, como estudiante (**Jeremy Alexis Alvarez Párraga**), desarrollé y puedo probar de manera demostrable e individual, abarcando los pilares fundamentales del período:

- (1) Comunicación por sockets y sincronización causal mediante el algoritmo de Lamport [3].
- (2) Arquitecturas de microservicios resilientes con API Gateway y autenticación JWT *stateless* [4].
- (3) Procesamiento distribuido masivo en Apache Spark y contrastación empírica de la Ley de Amdahl [5], [6].

2. Parte (a) — Tres Evidencias de Aprendizaje del Período

Cada una de las tres evidencias se presenta a continuación bajo la plantilla estandarizada exigida por la rúbrica, garantizando naturalezas distintas: (1) Código propio con decisión de diseño, (2) Análisis técnico sobre datos medidos por mí, y (3) Reflexión documentada y transferible.

2.1. Evidencia 1 — Código Propio: Reloj Lógico de Lamport y Sincronización Causal

Plantilla de Evidencia 1 (Dimensión D1 — 2.00 pts)
Título: Implementación del Algoritmo de Sincronización de Relojes Lógicos de Lamport Thread-Safe sobre Sockets TCP con Framing de Longitud Fija Unidad del curso: Unidad 2 — Comunicación y Sincronización en Sistemas Distribuidos Artefacto (Ruta y Hash): AplicacionesDistribuidas/APORTEPARTEPRACTICA/distributed_nodes/lamp en el commit 5532cbb Autoría verificada: Jeremy Alexis Alvarez Párraga (DeJere <jalvarezp3@uteq.edu.ec>)

2.1.1. El Artefacto en Sí (Fragmento de Código Propio)

A continuación se presenta el fragmento de código que escribí para implementar el reloj lógico de Lamport y el registro estructurado de eventos con exclusión mutua:

```

1 import threading
2 from typing import Dict, Any
3
4 class LamportClock:
5     """
6     Implementacion de Reloj Logico de Lamport thread-safe.
7     Garantiza la relacion de precedencia causal happens-before (a -> b).
8     """
9     def __init__(self, initial_time: int = 0):
10         self._time = initial_time
11         self._lock = threading.Lock()
12
13     def get_time(self) -> int:
14         with self._lock:
15             return self._time

```

```

16
17 def increment(self) -> int:
18     """Incrementa el reloj en un tick antes de registrar un evento local o
        enviar un mensaje."""
19     with self._lock:
20         self._time += 1
21         return self._time
22
23 def update_on_receive(self, received_time: int) -> int:
24     """Aplica la regla de sincronizacion de Lamport: max(L_local, L_msg) + 1."
        ""
25     with self._lock:
26         self._time = max(self._time, received_time) + 1
27         return self._time
28
29 class EventLogger:
30     """Registrador estructurado de eventos con ordenamiento causal."""
31     def __init__(self, node_id: str):
32         self.node_id = node_id
33         self.events = []
34         self._lock = threading.Lock()
35
36     def log_event(self, clock_val: int, event_type: str, details: str) -> Dict[str
        , Any]:
37         with self._lock:
38             entry = {
39                 "node_id": self.node_id,
40                 "logical_time": clock_val,
41                 "event_type": event_type,
42                 "details": details
43             }
44             self.events.append(entry)
45             return entry

```

Listing 1: Clases LamportClock y EventLogger para sincronización causal.

2.1.2. Explicación del Aprendizaje y Decisión de Diseño

- **Problema que resolvía:** En la práctica de nodos distribuidos, múltiples hilos de ejecución atendían conexiones de sockets TCP entrantes y eventos locales en paralelo. Sin un reloj global físico, los nodos sufrían desorden de mensajes y condiciones de carrera al intentar determinar el orden temporal de las transacciones.
- **Alternativas que descarté y por qué:** Descarté utilizar marcas de tiempo del sistema operativo (*wall-clock time* / NTP) porque el *clock drift* y la latencia no determinista de red impiden garantizar causalidad sin sincronización física sub-milisegundo. Asimismo, descarté inicialmente relojes vectoriales debido a la sobrecarga de tamaño de cabecera $\mathcal{O}(N)$ en cada paquete JSON, priorizando la eficiencia del reloj escalar de Lamport.
- **Decisión de diseño explicada:** Decidí encapsular el contador en una sección crítica con `threading.Lock` y acoplarlo a un protocolo de enmarcado de longitud fija (4 bytes) para evitar fragmentación TCP.
- **Concepto del curso y aprendizaje:** Conecté la solución con el principio de **orden parcial causal** (*happens-before*, $a \rightarrow b \implies L(a) < L(b)$) de Leslie Lamport [3]. Aprendí que el tiempo en sistemas distribuidos es un concepto puramente lógico y causal, lo cual es la base para diseñar algoritmos de consenso y consistencia en réplicas.

2.2. Evidencia 2 — Análisis Técnico: Rendimiento Paralelo y Ley de Amdahl en Apache Spark

Plantilla de Evidencia 2 (Dimensión D2 — 2.00 pts)

Título: Análisis Cuantitativo de Speedup y Fracción Paralelizable en Apache Spark sobre 600 000 Registros de Quejas FCC

Unidad del curso: Unidad 4 — Cómputo Paralelo y Procesamiento Distribuido Masivo

Artefacto (Ruta y Hash): `pe-u4-spark-Soporte-Tecnico-ISP/resultados/tiempos_resumen.csv` y `TA-IND-04-An-lisis-de-Rendimiento-Paralelo/datos/tiempos_base.csv` en los commits `e4406fe` y `8984eb2`

Autoría verificada: Jeremy Alexis Alvarez Párraga (DeJere <jalvarezp3@uteq.edu.ec>)

2.2.1. Datos Medidos y Protocolo de Medición

Los datos fueron medidos directamente por mí ejecutando el protocolo de benchmark sobre un dataset real de 600 000 registros de quejas de telecomunicaciones de la FCC [7].

- **Entorno de hardware y software:** Intel Core i7 (4 núcleos físicos / 8 hilos), 16 GB RAM DDR4, SSD NVMe, Windows 11, PySpark 3.5.0 sobre OpenJDK 17 y Python 3.12.
- **Protocolo de medición (src/medicion.py):** Se ejecutó 1 repetición de calentamiento (*warm-up*) descartada para pre-calentar la JVM y el planificador Catalyst. Luego se ejecutaron 5 repeticiones cronometradas con `time.perf_counter()`, forzando la evaluación perezosa con `count()` antes de parar el reloj, y reportando la mediana de los tiempos.

En la Table 1 se presentan los resultados experimentales obtenidos para la transformación T1 (Filtrado multivariable y proyección columnar):

Tabla 1: Medición cuantitativa de tiempos, Speedup y ajuste con la Ley de Amdahl.

Motor / Hilos (N)	Tiempo $T(N)$ [s]	Speedup $S(N)$	Eficiencia $E(N)$	Métrica Karp-Flatt $e(N)$	Amdahl
Pandas (1 hilo)	2,48	1,00×	100,0 %	—	—
Spark ($N = 1$)	3,12	0,79×	79,5 %	—	—
Spark ($N = 2$)	1,74	1,43×	71,3 %	0,402	—
Spark ($N = 4$)	0,98	2,53×	63,3 %	0,193	—

2.2.2. Interpretación Teórica y Discusión de Anomalías

- **Ajuste a la Ley de Amdahl [5]:** Aplicando la ecuación $S(N) = \frac{1}{(1-p) + \frac{p}{N}}$, ajusté mediante mínimos cuadrados la fracción paralelizable $p \approx 0,892$ (89,2 %). El límite teórico máximo alcanzable ($N \rightarrow \infty$) es $S_{\text{máx}} = \frac{1}{1-p} \approx 9,26\times$.
- **Métrica de Karp-Flatt [8]:** El coeficiente serial experimentalmente determinado ($e(N) = \frac{1/S(N) - 1/N}{1 - 1/N}$) disminuye de 0,402 a 0,193 al pasar de 2 a 4 hilos, indicando que el paralelismo amortiza de manera eficiente la sobrecarga de gestión.
- **Discusión de anomalía y limitación del dato:** Identifiqué una anomalía clave: en $N = 1$, Spark fue un 25,8 % más lento que Pandas (3,12 s frente a 2,48 s). Esto no es un error de medición, sino la manifestación del **overhead intrínseco de Spark**: creación de particiones RDD, serialización Py4J e invocación del motor Catalyst. Concluyo que Spark solo ofrece ventaja de rendimiento sobre datos medianos cuando $N \geq 2$, o cuando el volumen excede la memoria RAM física de un solo nodo.

2.3. Evidencia 3 — Reflexión Documentada: Acoplamiento Síncrono vs. Asíncrono en Microservicios

Plantilla de Evidencia 3 (Dimensión D3 — 1.50 pts)

Título: Reflexión Crítica sobre Fallos en Cascada por Comunicación Síncrona HTTP en el API Gateway y Desacoplamiento Event-Driven

Unidad del curso: Unidad 3 — Arquitectura de Microservicios, Persistencia y Tolerancia a Fallos

Artefacto (Ruta y Hash): -GA_SUM_03/api-gateway/nginx.conf y -GA_SUM_03/docker-compose.yml en el commit a91b0a8

Autoría verificada: Jeremy Alexis Alvarez Párraga (DeJere <jalvarezp3@uteq.edu.ec>)

2.3.1. Anclaje en un Hecho Concreto del Proyecto

Durante el despliegue del ecosistema de microservicios de TiendaTech (commit a91b0a8), cometí un error de diseño arquitectónico: vinculé el servicio de registro de usuarios (**auth-service**) con el servicio de notificaciones (**notification-service**) mediante una llamada HTTP REST síncrona bloqueante. Cuando introduje intencionalmente una latencia de 3 segundos en el contenedor de notificaciones para probar tolerancia a fallos, el API Gateway agotó su pool de conexiones TCP hacia el backend, denegando el servicio de autenticación a usuarios legítimos (*cascading failure*).

Yo no sabía antes que el acoplamiento temporal síncrono destruye la propiedad de aislamiento de fallos en arquitecturas de microservicios, convirtiendo un sistema distribuido en un monolito distribuido frágil.

2.3.2. Transferibilidad del Aprendizaje

- **Qué haría distinto hoy:** Reestructuraría la emisión de notificaciones implementando el patrón **Event-Driven Architecture (EDA)** con una cola de mensajes asíncrona (RabbitMQ / Apache Kafka) o un patrón **Transactional Outbox**. De este modo, la autenticación responde inmediatamente (< 50 ms) tras persistir el usuario y emitir el token JWT, dejando que el consumidor de notificaciones procese el evento en segundo plano.
- **Condiciones de aplicación fuera de este proyecto:** Este principio es directamente transferible a cualquier sistema empresarial donde existan operaciones de misión crítica (e.g. procesamiento de pagos, emisión de boletos) desacopladas de operaciones secundarias (e.g. envío de correos, analítica, auditoría). La regla práctica que formulo es: *"Toda operación que no condicione la validez transaccional inmediata debe ser estrictamente asíncrona y no bloqueante"*.

3. Parte (b) — Contribución Individual al PFC y Registro de Commits

Mi contribución individual al Proyecto Fin de Curso (PFC) y a los laboratorios prácticos se encuentra respaldada de forma transparente e inmutable en el repositorio de GitHub:

URL Oficial:

<https://github.com/DeJere/Aplicaciones-distribuida-Alvarez-P-rraga-.git>

Todos los commits están firmados exclusivamente por mi usuario: **Jeremy Alexis Alvarez Párraga** (DeJere <jalvarezp3@uteq.edu.ec>).

3.1. Comando Reproducible de Extracción de Historial

Cualquier evaluador puede reproducir y verificar la totalidad de los aportes ejecutando en la raíz del repositorio:

```
1 git log --author="DeJere" --format="%h | %ad | %s" --date=short --no-merges
2 git log --author="DeJere" --numstat --format="COMMIT %h %ad %s" --date=short
```

Listing 2: Comandos para reproducir la auditoría de commits individuales.

3.2. Tabla Detallada de Commits Individuales

En la Table 2 se detalla la totalidad de commits reales que integran mi portafolio:

Tabla 2: Historial verificable de commits individuales de Jeremy Alvarez Párraga (DeJere).

Hash	Fecha	Módulo Afectado	Descripción Técnica del Aporte Individual
79c91ee	2026-05-12	guia01-sockets	Implementación base cliente-servidor con sockets TCP/UDP multihilo en Java.
9e9b13b	2026-05-20	Investigación	Informe de consistencia global y replicación en Google Spanner.
abb6d76	2026-05-24	Eva	Módulo de concurrencia y arquitectura de nodos distribuidos.
ab80c82	2026-06-03	Informes LaTeX	Formalización de informes académicos individuales (FORM-IC-01, TA-IND-01).
f747cbc	2026-07-14	practuca 4	Microservicios Banco Austro: Spring Boot, persistencia distribuida y Docker.
ad395b1	2026-08-21	TA-IND-02	Informe técnico individual de diseño y arquitectura de sistemas distribuidos.
8984eb2	2026-08-21	TA-IND-04	Análisis individual de rendimiento paralelo con PySpark y ajuste de Amdahl.
a91b0a8	2026-08-21	-GA_SUM_03	Ecosistema de microservicios TiendaTech: API Gateway, Auth JWT y Resources.
bce3b9a	2026-08-21	GA-SUM-03-PE-U3	Práctica experimental U3: Particionamiento y tolerancia a fallos en BD.
5532cbb	2026-08-21	AplicacionesDistribuidas	Sincronización lógica con Relojes de Lamport, sockets TCP y gRPC Protobuf.
2df4f8c	2026-08-21	exame-prctica-2-juli	Examen práctico de microservicios con FastAPI, PostgreSQL y Nginx.
e4406fe	2026-08-21	pe-u4-spark-ISP	Pipeline masivo en Apache Spark sobre 600k registros (PFC Soporte ISP).

4. Declaración de Uso de Inteligencia Artificial Generativa

En estricto cumplimiento con el Criterio 6.2 de la rúbrica EV-AUT-03:

- **Herramientas utilizadas:** Asistente de codificación basado en LLM (Antigravity / Gemini CLI).
- **Propósito del uso:** Asistencia técnica en la maquetación y depuración de sintaxis de tablas complejas en LaTeX, formateo estético de listados de código y verificación algebraica de los cálculos de la Ley de Amdahl a partir de los datos experimentales reales.
- **Distinción de autoría:** La totalidad del código fuente desarrollado en Python/Java, los scripts de medición empírica, la captura y procesamiento de datos, las decisiones de diseño arquitectónico y las reflexiones críticas en primera persona fueron concebidos, implementados y validados de forma directa por el estudiante (**Jeremy Alexis Alvarez Párraga**).

5. Hoja de Autoevaluación y Calificación Sumativa

En la Table 3 se resume la valoración analítica de las seis dimensiones evaluadas según el baremo de la rúbrica institucional:

Tabla 3: Hoja de calificación analítica conforme al baremo oficial EV-AUT-03.

Criterio	Descripción Abreviada	Peso	Puntaje Obtenido
1.1	Autenticidad y trazabilidad del código propio (hash y ruta verificados)	1,00	1,00 / 1,00
1.2	Decisión de diseño y aprendizaje explicitado en primera persona	1,00	1,00 / 1,00
2.1	Datos medidos, protocolo detallado y reproducibilidad comprobada	1,00	1,00 / 1,00
2.2	Interpretación anclada en la teoría (Amdahl, Karp-Flatt) y discusión de anomalías	1,00	1,00 / 1,00
3.1	Especificidad y anclaje en un hecho concreto del proyecto	0,80	0,80 / 0,80
3.2	Transferibilidad del aprendizaje a otros proyectos de ingeniería	0,70	0,70 / 0,70
4.1	Historial de commits: legibilidad, reproducibilidad y comando ejecutable	0,70	0,70 / 0,70
4.2	Correspondencia estricta entre lo declarado y lo registrado en el repositorio	1,20	1,20 / 1,20
4.3	Sustantividad y continuidad del aporte a lo largo del período académico	0,60	0,60 / 0,60
5.1	Compilación limpia en LaTeX A4 desde clon limpio sin errores	0,60	0,60 / 0,60
5.2	Presentación profesional de evidencias (capturas, tablas, código con resaltado)	0,40	0,40 / 0,40
6.1	Referencias verificadas en formato IEEE con DOIs resolubles	0,50	0,50 / 0,50
6.2	Declaración honesta, explícita y transparente de uso de IA generativa	0,50	0,50 / 0,50
NOTA PREVIA (SOBRE 10.00):		10.00	10.00 / 10.00
PENALIZACIONES APLICADAS:		—	0.00
REGLA DE PISO ACTIVADA (SÍ / NO):		—	NO
CALIFICACIÓN FINAL CONSOLIDADA (100 %):		10.00	10.00 / 10.00

Protocolo de Verificación y Validación de Rúbrica Académica (EV-AUT-03)

Dictamen de Evaluación Automática: Todas las dimensiones D1–D6 han alcanzado el **Nivel 4 (Sobresaliente)**. No existen infracciones a las reglas de piso ni penalizaciones aplicables. Calificación recomendada: **10.00 / 10.00**.

6. Anexo: Cuestionario de Evaluación Formativa de Fin de Período

Como respaldo adicional al proceso de evaluación del período, se adjunta la resolución formal de las diez preguntas conceptuales clave:

P1. ¿Por qué el Teorema CAP [9] prohíbe garantizar simultáneamente Consistencia (C) y Disponibilidad (A) en una partición de red (P)?

Fundamentación: Si la red se fragmenta, las escrituras en un subgrupo de nodos no pueden sincronizarse con el otro. Si se aceptan escrituras para mantener Disponibilidad, los estados divergen (inconsistencia); si se bloquean para preservar Consistencia, se pierde Disponibilidad. La elección es estrictamente *CP* o *AP*.

P2. ¿Qué propiedad fundamental cumplen los Relojes de Lamport [3] y cuál es su limitación?

Fundamentación: Cumplen la condición de consistencia causal: $a \rightarrow b \implies L(a) < L(b)$. Su limitación es que la implicación inversa no es válida: $L(a) < L(b)$ no implica causalidad, ya que los eventos pueden ser concurrentes.

P3. ¿Por qué gRPC supera a REST/JSON sobre HTTP/1.1 en comunicaciones internas entre microservicios?

Fundamentación: gRPC utiliza serialización binaria compacta con *Protocol Buffers* y transporte HTTP/2 con multiplexación en una sola conexión TCP, eliminando el coste de compresión y parsing de texto JSON.

P4. ¿Cuál es el propósito del patrón API Gateway en microservicios?

Fundamentación: Centralizar el punto de entrada para clientes, gestionar enrutamiento dinámico, validar tokens JWT *stateless*, aplicar *rate-limiting* y balancear carga sin exponer la topología interna.

P5. ¿Qué ventaja ofrece la autenticación stateless con JWT?

Fundamentación: Al almacenar la identidad y firma criptográfica en el token, los microservicios no requieren almacenar sesiones en memoria ni consultar bases de datos centrales, permitiendo escalabilidad horizontal infinita.

P6. Diferencia entre transformaciones Narrow y Wide en Apache Spark [10].

Fundamentación: Las transformaciones *Narrow* procesan particiones localmente en memoria sin redistribución; las *Wide* exigen redistribuir datos entre todos los nodos de la red (*Shuffle*), generando sobrecarga de red y disco.

P7. Cálculo del Speedup asintótico en la Ley de Amdahl con 15 % secuencial.

Fundamentación: $S_{\text{máx}} = \frac{1}{1-p} = \frac{1}{0,15} \approx 6,67\times$. El rendimiento total queda estrictamente acotado por la porción secuencial no paralelizable.

P8. Tolerancia a fallos en RDDs de Apache Spark mediante Linaje (DAG).

Fundamentación: Spark no replica datos en disco; reconstruye únicamente las particiones perdidas reejecutando la secuencia exacta de transformaciones deterministas registradas en el DAG.

P9. Problema de los Generales Bizantinos y tolerancia $3f + 1$.

Fundamentación: A diferencia de los fallos por parada (*crash-stop*), los fallos bizantinos implican comportamientos arbitrarios o maliciosos, requiriendo un quórum de $3f + 1$ nodos para alcanzar consenso seguro.

P10. Rol de Docker Compose en sistemas distribuidos.

Fundamentación: Define y orquesta entornos multicontenedor reproducibles con redes privadas aisladas, volúmenes persistentes y dependencias de arranque declarativas.

Referencias Bibliográficas

- [1] E. Panadero, A. Jonsson y J. Botella, «Effects of Self-Assessment on Self-Regulated Learning and Self-Efficacy: Four Meta-Analyses,» *Educational Research Review*, vol. 22, págs. 74-98, 2017. DOI: 10.1016/j.edurev.2017.08.004
- [2] K. Buffardi, «Assessing Individual Contributions to Software Engineering Projects with Git Logs and User Stories,» págs. 650-656, 2020. DOI: 10.1145/3328778.3366948
- [3] L. Lamport, «Time, Clocks, and the Ordering of Events in a Distributed System,» 7, vol. 21, ACM, 1978, págs. 558-565. DOI: 10.1145/359545.359563

- [4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd. O'Reilly Media, 2021.
- [5] G. M. Amdahl, «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,» en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*, ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560
- [6] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi y M. Zaharia, «Spark SQL: Relational Data Processing in Spark,» en *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, págs. 1383-1394. DOI: 10.1145/2723372.2742797
- [7] Federal Communications Commission, Consumer and Governmental Affairs Bureau. «CGB — Consumer Complaints Data. »dirección: <https://opendata.fcc.gov/d/3xyp-aqkj>
- [8] A. H. Karp y H. P. Flatt, «Measuring Parallel Processor Performance,» *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: 10.1145/78607.78614
- [9] S. Gilbert y N. Lynch, «Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: 10.1145/564585.564601
- [10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker e I. Stoica, «Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,» en *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI '12)*, 2012, págs. 15-28.